

Phase 4: MCP Server & Chat Interface

POC-07 — Inventory Management & Procurement System

Phase Weight: 25% | **Duration:** 5 working days | **Test Cases:** 25

1. Phase Overview

Convert the Phase 1 REST API into an MCP server with 6 tools, then build a conversational Streamlit chat interface for inventory management queries.

2. MCP Server (mcp_server/mcp_app.py)

```
import requests
import structlog
from fastmcp import FastMCP
from opentelemetry import trace

logger = structlog.get_logger()
tracer = trace.get_tracer("poc-07-mcp")
BASE_URL = "http://localhost:8000/api/v1"
mcp = FastMCP("Inventory Management Server")

def _get(path, params=None):
    try:
        r = requests.get(f"{BASE_URL}{path}", params=params or {}, timeout=10)
        r.raise_for_status()
        return r.json()
    except requests.exceptions.ConnectionError:
        return {"error": "API unavailable"}
    except Exception as e:
        return {"error": str(e)}

def _post(path, payload):
    try:
        r = requests.post(f"{BASE_URL}{path}", json=payload, timeout=10)
        r.raise_for_status()
        return r.json()
    except requests.exceptions.ConnectionError:
        return {"error": "API unavailable"}
    except Exception as e:
        return {"error": str(e)}

@mcp.tool()
def update_stock(product_id: int, movement_type: str, quantity: int,
                 reference_number: str = None, notes: str = None) -> dict:
    """Update stock for a product by recording a stock movement. movement_type
    must be one of:
```

receipt, sale, adjustment, transfer, return. Positive quantity = stock in, negative = stock out.

Automatically triggers low_stock alert if available quantity drops below reorder point."""

```
with tracer.start_as_current_span("mcp.tool.update_stock") as span:
    span.set_attribute("poc_id", "POC-07")
    payload = {"movement_type": movement_type, "quantity": quantity,
               "reference_number": reference_number, "notes": notes}
    result = _post(f"/products/{product_id}/stock", payload)
    logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
               tool="update_stock", product_id=product_id)
    return result
```

@mcp.tool()

```
def create_purchase_order(supplier_id: int, order_date: str,
                          items: list, expected_delivery: str = None) -> dict:
    """Create a new purchase order for a supplier. items should be a list of dicts
    with
    product_id, quantity_ordered, unit_cost. PO number is auto-generated as PO-
    YEAR-NNNN.
```

Initial status is 'draft'."""

```
with tracer.start_as_current_span("mcp.tool.create_purchase_order") as span:
    span.set_attribute("poc_id", "POC-07")
    payload = {"supplier_id": supplier_id, "order_date": order_date,
               "items": items, "expected_delivery": expected_delivery}
    result = _post("/orders", payload)
    logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
               tool="create_purchase_order", supplier_id=supplier_id)
    return result
```

@mcp.tool()

```
def get_low_stock_products() -> list:
```

"""Get all products currently at or below their reorder point. Returns products needing

immediate procurement attention, sorted by urgency (out-of-stock first, then lowest

available quantity). Use this to prioritize purchase order creation."""

```
with tracer.start_as_current_span("mcp.tool.get_low_stock_products") as span:
    span.set_attribute("poc_id", "POC-07")
    result = _get("/stock/low-alerts")
    logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
               tool="get_low_stock_products")
    return result
```

@mcp.tool()

```
def get_supplier_catalog(supplier_id: int) -> list:
```

"""Get a supplier's product catalog with SKUs, product names, and cost prices. Use when creating purchase orders or comparing supplier prices for procurement decisions."""

```
with tracer.start_as_current_span("mcp.tool.get_supplier_catalog") as span:
    span.set_attribute("poc_id", "POC-07")
    result = _get(f"/suppliers/{supplier_id}/catalog")
    logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
               tool="get_supplier_catalog", supplier_id=supplier_id)
```

```

        return result

@mcp.tool()
def get_purchase_orders(status: str = None, supplier_id: int = None) -> list:
    """List purchase orders with optional filtering by status
    (draft/submitted/acknowledged/received/cancelled)
    or supplier. Returns PO number, supplier, status, total amount, and expected
    delivery date."""
    with tracer.start_as_current_span("mcp.tool.get_purchase_orders") as span:
        span.set_attribute("poc_id", "POC-07")
        params = {}
        if status:
            params["status"] = status
        if supplier_id:
            params["supplier_id"] = supplier_id
        result = _get("/orders", params)
        logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
                    tool="get_purchase_orders")
        return result

@mcp.tool()
def get_inventory_dashboard() -> dict:
    """Get the inventory management dashboard with overall health metrics: total
    products,
    low stock count, out of stock count, open purchase orders, and total stock
    value in ₹."""
    with tracer.start_as_current_span("mcp.tool.get_inventory_dashboard") as span:
        span.set_attribute("poc_id", "POC-07")
        result = _get("/dashboard")
        logger.info("mcp_tool_called", poc_id="POC-07", phase="P4",
                    tool="get_inventory_dashboard")
        return result

if __name__ == "__main__":
    mcp.run()

```

3. Chat Interface (mcp_server/chat_interface.py)

```

import structlog
from typing import List, Dict
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain import hub
from langchain.tools import StructuredTool
from langsmith import traceable
from mcp_server.mcp_app import (update_stock, create_purchase_order,
                                get_low_stock_products,
                                get_supplier_catalog, get_purchase_orders,
                                get_inventory_dashboard)

```

```

logger = structlog.get_logger()

class ChatSession:
    def __init__(self, session_id: str):
        self.session_id = session_id
        self.history: List[Dict[str, str]] = []

    def add_message(self, role: str, content: str):
        self.history.append({"role": role, "content": content})

    def get_history_for_llm(self) -> List[Dict[str, str]]:
        return self.history[-10:]

def build_chat_executor() -> AgentExecutor:
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.1)
    tools = [
        StructuredTool.from_function(update_stock, description="Record a stock
movement for a product"),
        StructuredTool.from_function(create_purchase_order, description="Create a
purchase order for a supplier"),
        StructuredTool.from_function(get_low_stock_products, description="Get all
low stock and out-of-stock products"),
        StructuredTool.from_function(get_supplier_catalog, description="Get
supplier product catalog with prices"),
        StructuredTool.from_function(get_purchase_orders, description="List
purchase orders by status or supplier"),
        StructuredTool.from_function(get_inventory_dashboard, description="Get
inventory dashboard metrics"),
    ]
    prompt = hub.pull("hwchase17/react")
    agent = create_react_agent(llm, tools, prompt)
    return AgentExecutor(agent=agent, tools=tools, verbose=True,
                        handle_parsing_errors=True, max_iterations=5)

@traceable(project_name="AI-Readiness-POC-07-P4")
def process_message(message: str, session_id: str = "default") -> dict:
    try:
        executor = build_chat_executor()
        logger.info("chat_message", poc_id="POC-07", phase="P4",
                    session_id=session_id, message_preview=message[:60])
        result = executor.invoke({"input": message})
        return {"output": result.get("output", ""), "session_id": session_id}
    except Exception as e:
        logger.error("chat_error", poc_id="POC-07", phase="P4", error=str(e))
        return {"output": f"Error: {str(e)}", "session_id": session_id}

```

4. MCP Tool Reference

Tool	Phase 1 Endpoint	Description
<code>update_stock</code>	<code>PATCH /products/{id}/stock</code>	Record stock movement

Tool	Phase 1 Endpoint	Description
<code>create_purchase_order</code>	POST /orders	Create PO
<code>get_low_stock_products</code>	GET /stock/low-alerts	Low stock list
<code>get_supplier_catalog</code>	GET /suppliers/{id}/catalog	Supplier products
<code>get_purchase_orders</code>	GET /orders	PO list with filters
<code>get_inventory_dashboard</code>	GET /dashboard	Health metrics

LangSmith project: `AI-Readiness-POC-07-P4`

5. Submission Checklist

- ☐ `mcp = FastMCP("Inventory Management Server")`
- ☐ All 6 tools defined with `@mcp.tool()`
- ☐ `get_inventory_dashboard` returns `{"error": ...}` on connection failure
- ☐ `ChatSession` with `history`, `add_message`, `get_history_for_llm`
- ☐ `build_chat_executor()` returns executor with 6 tools
- ☐ 25 test cases: ≥ 18 passing